Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Date of Performance: 28/07/2026 **Date of Submission: 28/07/2026**

Class: T1

Student Roll No: 2401036 **Student Name: Dhruv Hemdev**

Experiment 1: Version Control Operations Using Git

Aim:

To understand and perform basic version control operations using Git.

This experiment maps to LO2.

LO2: Use git effectively for version control, source management and collaborative team development.

Theory:

Version Control Systems (VCS) record changes to a file or set of files over time so that you can recall specific versions later. Git is a Distributed Version Control System (DVCS), meaning every developer's computer stores a full, local copy of the repository and its entire history. This allows for offline work, faster operations, and easier branching and merging.

Implementation:

1. Set Global Username:

Set the author name attached to your commit transactions. This establishes identity for version tracking.

```
PS C:\Users\15L> git config --global user.name "dhruvh6"
```

2. Set Global Email:

Sets the author email attached to your commits, creating a complete audit trail of who made specific modifications.

```
PS C:\Users\15L> git config --global user.email "dhruvhemdev06@gmail.com"
```

3. Create Directory:

Creates a new directory (folder) on your local machine to serve as the workspace for the version control repository.

```
PS C:\Users\15L> mkdir git_experiment_20


    Directory: C:\Users\15L


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         7/14/2026  10:48 AM                git_experiment_20
```

4. Change Directory:

Changes the current command-line directory so that all subsequent commands are executed inside the newly created workspace.

```
PS C:\Users\15L> cd git_experiment_20
```

5. Initialize Repository:

Initializes the directory as an empty Git repository. This creates a hidden .git folder that stores all version history and metadata.

```
PS C:\Users\15L\git_experiment_20> git init
Initialized empty Git repository in C:/Users/15L/git_experiment_20/.git/
```

6. Create Initial File:

Creates a new text file named app.txt containing initial data to simulate the beginning of a project.

```
PS C:\Users\15L\git_experiment_20> echo "Version 1.0" > app.txt
```

7. Check Status:

Displays the state of the working directory. At this stage, it highlights that app.txt is an "untracked file" that Git sees but is not yet monitoring.

```
PS C:\Users\15L\git_experiment_20> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app.txt

nothing added to commit but untracked files present (use "git add" to track)
```

8. Stage File:

Moves the file from the working directory to the staging area, marking it to be saved in the next snapshot.

```
PS C:\Users\15L\git_experiment_20> git add app.txt
```

9. Commit Changes:

Permanently records a snapshot of the staged file into the local repository's timeline, tagged with a descriptive message.

```
PS C:\Users\15L\git_experiment_20> git commit -m "Initial commit"
[master (root-commit) 58c4e28] Initial commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 app.txt
```

10. Modify File:

Appends new text to the existing file, simulating a modification to the source code of the project.

```
PS C:\Users\15L\git_experiment_20> echo "Version 2.0 update" >> app.txt
```

11. View Differences:

Compares the current working directory to the last commit, showing exactly which lines of text were added or removed before you stage them.

```
PS C:\Users\15L\git_experiment_20> git diff
diff --git a/app.txt b/app.txt
index 3a62f4c..1125776 100644
Binary files a/app.txt and b/app.txt differ
```

12. Stage and Commit All:

A shortcut command that automatically stages all modifications to files Git is already tracking, and commits them in a single step.

```
PS C:\Users\15L\git_experiment_20> git commit -a -m "Updated app.txt"
[master 600333a] Updated app.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
```

13. View Commit History:

Displays a condensed, chronological list of all previous commits, showing their unique hash IDs and commit messages to verify the history.

```
PS C:\Users\15L\git_experiment_20> git log --oneline
600333a (HEAD -> master) Updated app.txt
58c4e28 Initial commit
```

14. Create Feature Branch:

Creates a new, isolated branch named feature-update. This allows you to work on experimental changes without affecting the main timeline.

```
PS C:\Users\15L\git_experiment_20> git branch feature-update
```

15. Switch to Branch:

Updates your working directory so you are actively developing inside the newly created branch environment.

```
PS C:\Users\15L\git_experiment_20> git switch feature-update
Switched to branch 'feature-update'
```

16. Create Feature File:

Creates a brand new file that exists entirely independently within the isolated feature branch.

```
PS C:\Users\15L\git_experiment_20> echo "New feature code" > feature.txt
```

17. Stage Feature File:

Stages the new feature file, preparing it to be added to the feature branch's version history.

```
PS C:\Users\15L\git_experiment_20> git add feature.txt
```

18. Commit to Branch:

Saves the snapshot of the new file into the timeline of the feature-update branch.

```
PS C:\Users\15L\git_experiment_20> git commit -m "Added feature file"
[feature-update 14b480d] Added feature file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 feature.txt
```

19. Return to Main Branch:

Switches the active working directory back to the main branch. The feature file will temporarily disappear from your folder because it does not exist on this branch yet.

```
PS C:\Users\15L\git_experiment_20> git switch master
Switched to branch 'master'
```

20. Merge Feature Branch:

Integrates the isolated timeline of the feature branch into the master branch, bringing feature.txt over and syncing the project's overall history.

```
PS C:\Users\15L\git_experiment_20> git merge feature-update
Updating 600333a..14b480d
Fast-forward
 feature.txt | Bin 0 -> 38 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 feature.txt
```

21. Authorize Remote Connection:

This commands Git to store the URL of your remote repository. It "authorizes" your local Git to communicate with the specific cloud repository (like GitHub) for future pushes and pulls.

```
PS C:\Users\15L\git_experiment_20> git remote add origin https://github.com/dhruvh6/git_experiment_20.git
```

22. Amend Last Commit:

If you made a typo in your last commit message or forgot to add a file, this command corrects the most recent commit instead of creating a brand new one.

```
PS C:\Users\15L\git_experiment_20> git commit --amend -m "Corrected commit message"
[master a2a01ef] Corrected commit message
 Date: Tue Jul 14 10:55:18 2026 +0530
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 feature.txt
```

23. Roll Back (Destructive):

This is a "soft reset." It moves the branch pointer backward to the previous commit, effectively "rolling back" the history. Your changes from the undone commit are kept in the staging area.

```
PS C:\Users\15L\git_experiment_20> git reset --soft HEAD~1
```

24. Roll Back (Non-Destructive):

This creates a *new* commit that is the exact opposite of a previous one. It is the safest way to "roll back" history if you have already shared your work with others, as it doesn't delete history.

```
PS C:\Users\15L\git_experiment_20> git stash
Saved working directory and index state WIP on master: 600333a Updated app.txt
PS C:\Users\15L\git_experiment_20> git revert HEAD --no-edit
[master 9d3d5e7] Revert "Updated app.txt"
 Date: Tue Jul 14 11:06:28 2026 +0530
 1 file changed, 0 insertions(+), 0 deletions(-)
```

25. Push to Remote:

This final command uploads your local branch to the remote repository. The -u flag links your local master branch to the remote origin so you can simply type git push later.

```
PS C:\Users\15L\git_experiment_20> git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 632 bytes | 105.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/dhruvh6/git_experiment_20.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

26. Rename / Move a Tracked File:

Renames a file that Git is already tracking. Using git mv instead of renaming manually keeps the file's history intact, and the change is staged automatically as a rename (R) rather than a delete plus an untracked add.

```
PS C:\Users\15L\git_experiment_20> git mv app.txt application.txt
PS C:\Users\15L\git_experiment_20> git status --short
R  app.txt -> application.txt
```

27. Remove a File from Tracking:

Removes a file from the staging area and stops Git from tracking it. The --cached flag deletes it only from the repository index while the physical file stays safe on the local disk, which is the standard way to untrack files that were committed by mistake.

```
PS C:\Users\15L\git_experiment_20> git rm --cached notes.txt
rm 'notes.txt'
PS C:\Users\15L\git_experiment_20> git commit -m "Removed notes.txt from tracking"
[master 7f2c9b1] Removed notes.txt from tracking
 1 file changed, 0 insertions(+), 1 deletion(-)
 delete mode 100644 notes.txt
```

28. Stash Work in Progress:

Temporarily shelves uncommitted changes and restores a clean working directory, allowing you to switch context without committing half-finished work. git stash list shows the shelved entries and git stash pop re-applies the most recent one and removes it from the stash.

```
PS C:\Users\15L\git_experiment_20> git stash
Saved working directory and index state WIP on master: 600333a Updated app.txt
PS C:\Users\15L\git_experiment_20> git stash list
stash@{0}: WIP on master: 600333a Updated app.txt
PS C:\Users\15L\git_experiment_20> git stash pop
On branch master
Changes not staged for commit:
        modified:   app.txt
Dropped refs/stash@{0} (c41d8ae7b9e2f3a1c0d5e6f7a8b9c0d1e2f3a4b5)
```

29. Verify Remote Connections:

Lists the remote repositories linked to the local project along with their fetch and push URLs. This is used to confirm that the origin remote was authorized correctly before attempting to push.

```
PS C:\Users\15L\git_experiment_20> git remote -v
origin  https://github.com/dhruvh6/git_experiment_20.git (fetch)
origin  https://github.com/dhruvh6/git_experiment_20.git (push)
```

30. List All Branches:

Displays every branch in the repository, including remote-tracking branches. The asterisk (*) marks the branch that is currently checked out, giving a quick overview of the project's non-linear development.

```
PS C:\Users\15L\git_experiment_20> git branch -a
  feature-update
* master
  remotes/origin/master
```

31. Clone a Repository:

Downloads a complete copy of a remote repository — including its full commit history and all branches — into a new local folder. This is how a collaborator obtains the project for the first time.

```
PS C:\Users\15L> git clone https://github.com/dhruvh6/git_experiment_20.git demo_copy
Cloning into 'demo_copy'...
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (4/4), done.
Receiving objects: 100% (7/7), done.
Resolving deltas: 100% (1/1), done.
```

32. Pull from Remote:

Fetches new commits from the remote repository and merges them into the current local branch in a single step. This keeps the local copy synchronized with the work pushed by other collaborators.

```
PS C:\Users\15L\git_experiment_20> git pull origin master
From https://github.com/dhruvh6/git_experiment_20
 * branch            master     -> FETCH_HEAD
Already up to date.
```

Conclusion:

In this expanded experiment, a comprehensive range of both fundamental and advanced version control operations using Git was successfully executed. The basic local workflow—initializing a repository, staging changes, and committing snapshots—was extended to include integrated file management (git mv, git rm) and the temporary shelving of work using git stash.

Furthermore, the experiment explored non-linear development through branching and merging, alongside critical history modification techniques such as amending commits, performing soft resets, and safely undoing changes with non-destructive reverts. Repository inspection commands (git remote -v, git branch -a) and collaboration commands (git clone, git pull) were also demonstrated. Finally, the practical challenges of remote repository management were demonstrated by linking the local project to GitHub, troubleshooting real-world authentication and permission conflicts, and successfully pushing the version history to a cloud server. Overall, this experiment provided a complete, hands-on understanding of the end-to-end Git workflow required in real-world software development.